

MICROPROCESSOR AND EMBEDDED SYSTEMS



PROJECT REPORT

SMART TRAFFIC LIGHT SYSTEM

NAME	NAUFAL SUL'AIM BIN MUHAMAD KHAIRI
CLASS	L05 B01
PROJECT NAME	SMART TRAFFIC LIGHT SYSTEM
LECTURER NAME	KAMALUL FAIZIN MOHD SUHUT

CONTENT

NO.	TITLE	PAGE
1	Project Objective	3
2	Operational Of the Application	4 – 5
3	Block Diagram	6
4	Flowchart	7
5	List Of Components	8
6	List Input/Output	9
7	Result	10
8	Coding Txt.	11 - 37
9	Conclusion	38

PROJECT OBJECTIVES

This project's goal is to design and implement a smart traffic system with the STM32 F401RE microcontroller. The system's goal is to improve traffic flow, increase safety, and prioritise pedestrian crossings at two intersections. The project also includes a timer display that uses a 7-segment component to graphically depict the remaining time for each traffic light phase.

I hope that by building this smart traffic system, People be able to overcome the shortcomings of traditional traffic management systems and deliver a more efficient and safe traffic control solution.

The following are the project's primary objectives:

1. **Improved Traffic Flow:** Using ultrasonic sensors, the smart traffic system will check the distance of vehicles approaching junctions. The technology can optimise traffic light timings by correctly recognising the presence of vehicles, decreasing congestion and delays at intersections.
2. **Enhanced Safety:** By integrating a pedestrian button as an interrupt, the system will prioritise pedestrian crossings. When a pedestrian requests a crossing, the system ensures their safety by coordinating with traffic signals to provide a safe passage.
3. **Timer Display:** A 7-segment display will be added into the system to improve visibility and understanding of traffic light timings. The display will show the remaining time for each traffic light phase, helping both cars and pedestrians to understand how much time is available for each light phase.
4. **Microcontroller-Based Implementation:** The system's control unit will be the STM32 F401RE microcontroller. It provides the processing power, GPIO ports, and communication interfaces required to communicate with various components such as the ultrasonic sensors, pedestrian button, traffic lights, and 7-segment display.

The initiative hopes to contribute to a more efficient and safer traffic management system by lowering traffic congestion, improving pedestrian safety, and improving overall traffic flow.

OPERATIONAL OF THE APPLICATION

Functional Definition

The functional definition outlines the specific features and operations of the smart traffic system.

1. Vehicle Detection

At one junction, the smart traffic system detects oncoming vehicles using ultrasonic sensors. The ultrasound sensors generate ultrasonic waves and time how long it takes for the waves to bounce back after colliding with an item. The technology can determine the presence and distance of cars by calculating closeness based on measured time. This data is critical for traffic control choices and optimising traffic light timing.

2. Traffic Light Control

The traffic light control capability is in duty of managing the traffic light signals at each junction based on the detection of vehicles and pedestrian crossing requests. To receive data from the ultrasonic sensors, the system employs the STM32 F401RE microcontroller. It uses sensor data to determine the presence of vehicles at roads.

The traffic light control algorithm optimises the timing of the traffic lights based on vehicle detection to ensure smooth traffic flow. If no vehicles were detected at a junction, the system can alter the light phases to eliminate unnecessary waiting and congestion. The traffic light signals are controlled by the microcontroller by activating the GPIO pins attached to the LEDs depicting green and red lights at each intersection.

3. Pedestrian Crossing

The smart traffic system has a pedestrian button as an interrupt to focus on pedestrian safety. When a pedestrian wishes to cross the path of traffic, they can do so by pressing the pedestrian button, which causes an interrupt signal to be generated. When the interrupt signal is received, the microcontroller interacts with the traffic lights to offer a dedicated pedestrian crossing phase.

When the pedestrian button is pushed, the traffic light control algorithm modifies the timing of the traffic lights to stop vehicle traffic and allow people to cross the road safely. The technique ensures that pedestrians have enough time to cross, and that vehicular traffic is stopped until the pedestrian crossing phase is finished.

4. *Timer Display*

The timer display feature improves visibility and understanding of traffic light timings. To visually depict the remaining time for each traffic light phase, the system includes a 7-segment display coupled to relevant GPIO pins on the microcontroller.

The microcontroller continuously updates the 7-segment display, representing the duration of each phase. The remaining time is converted into a suitable format and displayed on the 7-segment display. This provides a clear and simple display of the time remaining for a certain traffic signal phase, assisting both cars and pedestrians in planning their activities accordingly.

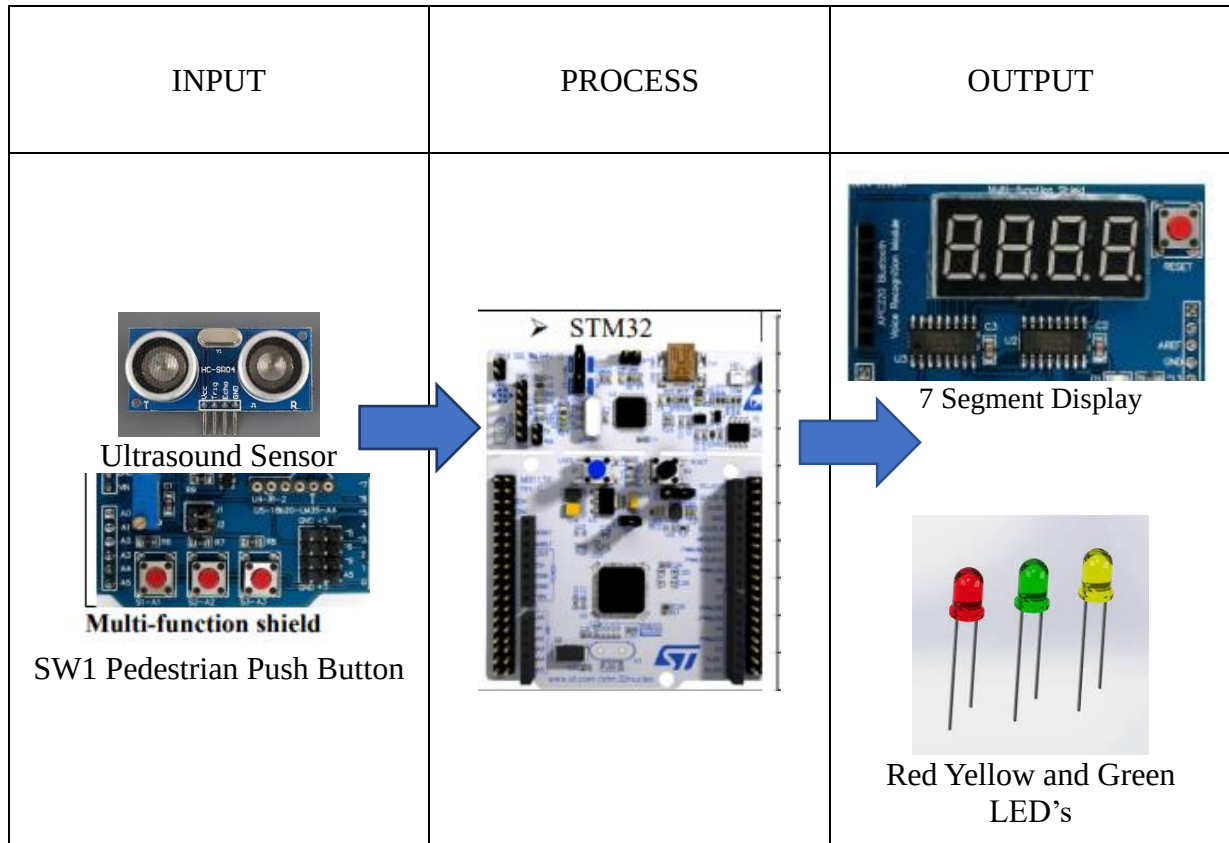
5. *Microcontroller Control*

The STM32 F401RE microcontroller serves as the central control unit of the smart traffic system. It handles the configuration of GPIO pins for sensor inputs (ultrasound sensors), control outputs (traffic lights and 7-segment display), and interrupt handling for the pedestrian button.

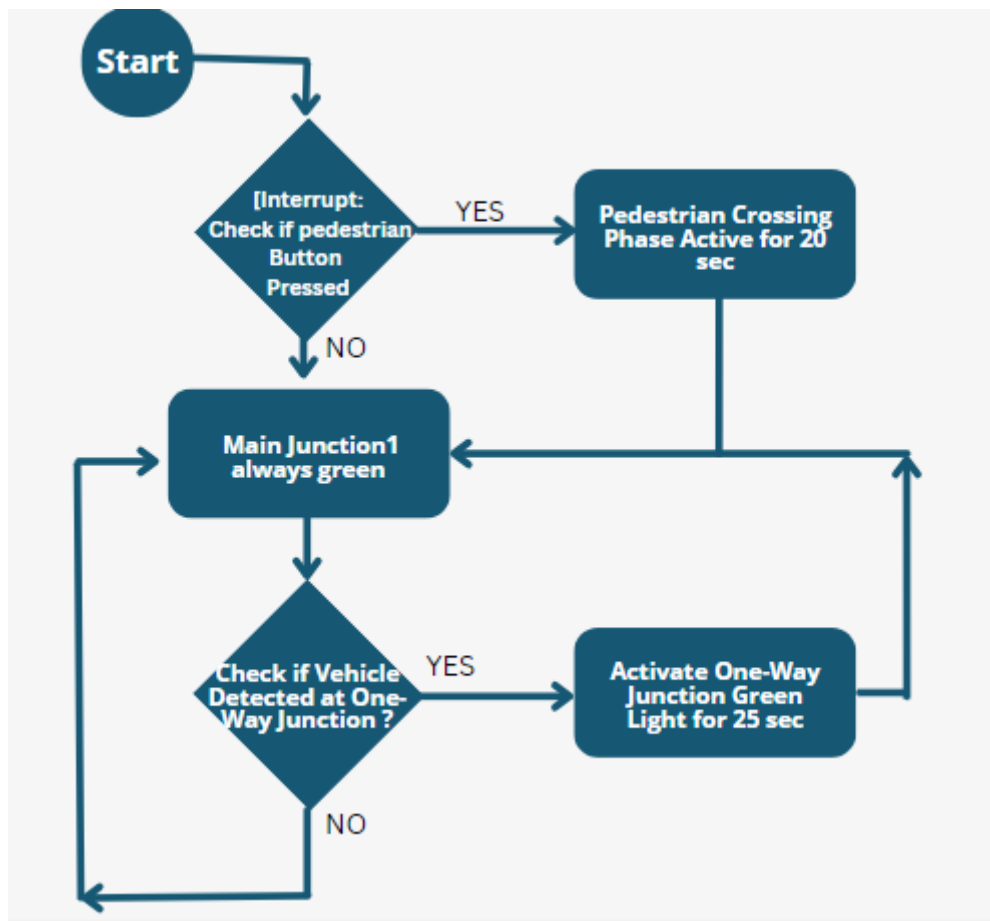
The microcontroller runs the traffic control algorithm, coordinating the inputs from the ultrasound sensors, pedestrian button, and timer display functionality. It manages the timing and synchronization of traffic light signals, updates the 7-segment display with the remaining time, and ensures the smooth operation of the entire system.

The functional definition highlights the specific features and operations of the smart traffic system, including vehicle detection, traffic light control, pedestrian crossing prioritization, timer display, and microcontroller control. These functions work together to optimize traffic flow, enhance safety, and provide clear and visible information to both drivers and pedestrians.

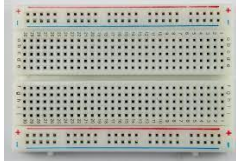
BLOCK DIAGRAM



PROJECT FLOWCHART



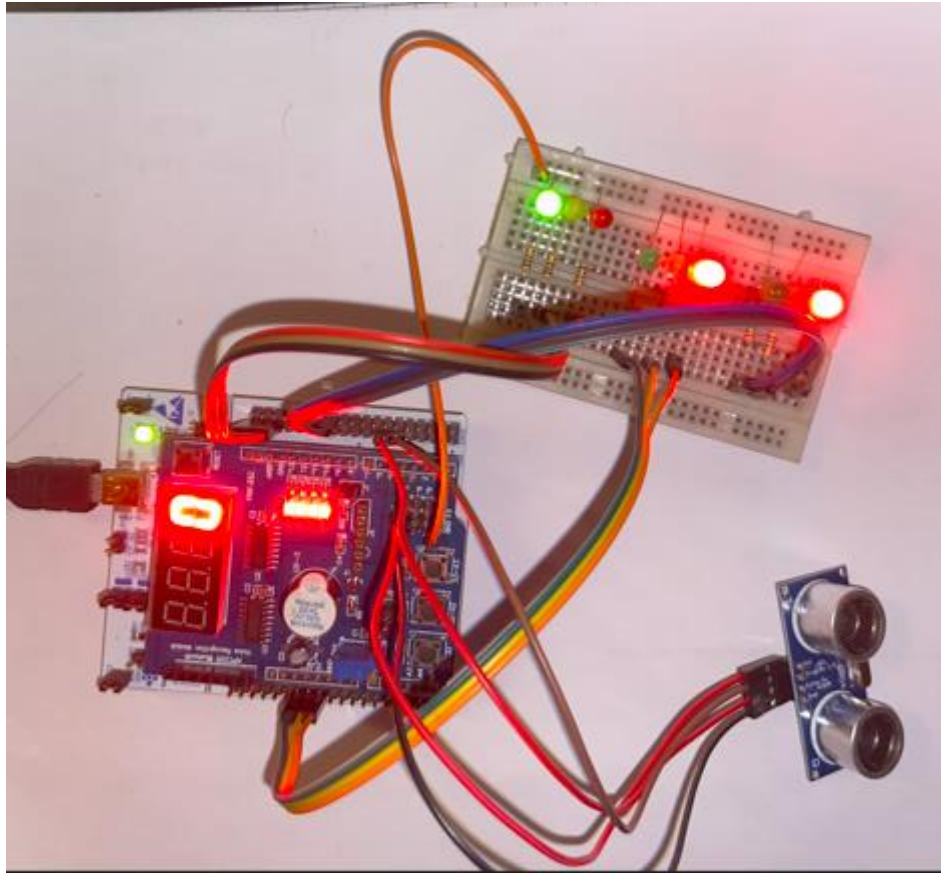
List Of Components

No.	Component's Name	Quantity	Picture
1	STM32 F401RE Microcontroller	1	
2	Ultrasound Sensors	1	
3	Pedestrian Button SW1	1	
4	7-Segment Display	1	
5	Green, Yellow and red LEDs	3 for each colour	
6	Resistors 150ohm	6 pcs	
7	Breadboard	1 pcs	
8	Jumper wires	16 pcs	

List Input/Output

No.	Component	Pin	Configuration
1	LED Green LED Yellow LED Red	PC 9, PA 12, PA 13 PB 8, PA 11, PA 14 PB 9, PB 12, PA 15	Output Push-pull
2	7 Segment Display	PA 9 Output PA 8 Output PB5 Output	Data In Serial Clock Parallel Clock
3	Pedestrian Push Button	PA 1 Input	Interrupt
4	Ultrasound Sensor	PB 4 SDA PB 10 SCL	(TRIG) I2C 1 (ECHO) I2C 1
5	Serial PC	USBTX, USBRX	

RESULT



CODING

```
#include "mbed.h"
#include "hcsr04.h"
// pins for ultrasound sensor
HCSR04 sensor(PB_4, PB_10);

// pins for pedestrian button interrupt
InterruptIn pedestrianButton(PA_1); // Pedestrian button pin (GPIO interrupt)

//USB serial communication
Serial pc(USBTX, USBRX); // Define Serial object for USB communication

Timer carTimer; // Timer object to track the duration of the obstacle
detection

// pins for traffic lights
DigitalOut junction1Green(PC_9); // Green LED for junction 1
DigitalOut junction1Yellow(PB_8); // Yellow LED for junction 1
DigitalOut junction1Red(PB_9); // Red LED for junction 1
DigitalOut junction2Green(PA_13); // Green LED for junction 2
DigitalOut junction2Yellow(PA_14); // Yellow LED for junction 2
DigitalOut junction2Red(PA_15); // Red LED for junction 2
DigitalOut PedestrianGreen1(PA_12); // GreenPedestrian LED for Pedetrian
DigitalOut PedestrianGreen2(PA_11); // GreenPedetrian LED for Pedestrian
DigitalOut PedestrianRed(PB_12); // RedPedestrian LED for Pedestrian

//Output for 7 segmment Counter Display
DigitalOut dataIn(PA_9);
DigitalOut serialClk(PA_8);
DigitalOut parallelClk(PB_5);
DigitalOut displayD1(PA_5);
DigitalOut displayD2(PA_6);
DigitalOut displayD3(PA_7);
DigitalOut displayD4(PB_6);

int num[11] = {0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80, 0x90,
0xFF};
int seg[4] = {0xF1, 0xF2, 0xF4, 0xF8};

int cnt = 0;
int16_t counter = 0;
int8_t d1, d2, d3, d4;

void LEDs(uint8_t x)
{
    displayD1 = (x & (1 << 0)) ? 1 : 0;
    displayD2 = (x & (1 << 1)) ? 1 : 0;
```

```

    displayD3 = (x & (1 << 2)) ? 1 : 0;
    displayD4 = (x & (1 << 3)) ? 1 : 0;
}

void ledcnt(void)
{
    LEDs(cnt);
    wait_ms(500);
    if (++cnt > 15)
        cnt = 0;
}

void displaySeg(uint8_t data, uint8_t seg)
{
    int8_t i;
    uint16_t dataseg;
    dataseg = (data << 8) + seg;

    parallelClk = 0;
    for (i = 15; i >= 0; i--)
    {
        serialClk = 0;
        if (dataseg & (1 << i))
            dataIn = 1;
        else
            dataIn = 0;
        serialClk = 1;
    }
    parallelClk = 1;
    wait_ms(1);
}

// Define variables
bool pedestrianCrossingActive = false; // Flag to track pedestrian crossing
phase

void CarDetection()
{
    long distance = sensor.distance(); // Get the distance from the
    ultrasonic sensor
    pc.printf("Distance: %ld cm\n", distance);

    if (distance < 7) {
        pc.printf("Car detected\n\r");
        // Additional code for car detection action
    } else {
        pc.printf("No car detected\n\r");
    }
}

```

```

    // Additional code for no car detection action
}

    wait(1);
}

void TrafficJunction1Offf()

{

    pedestrianButton.disable_irq();

    counter = 179;

    while (counter > 170)
    {
        d1 = (counter / 10) % 10;
        d2 = (counter / 100) % 10;
        d3 = (counter / 1000) % 10;
        d4 = (counter / 10000) % 10;
        for (int x = 0; x < 20; x++)
        {
            displaySeg(num[d4], seg[0]);
            displaySeg(num[d3], seg[1]);
            displaySeg(num[d2], seg[2]);
            displaySeg(num[d1], seg[3]);

            junction1Green = 0;
            junction1Yellow = 0;
            junction1Red = 0;

            junction2Green = 0;
            junction2Yellow = 0;
            junction2Red = 1;

            PedestrianGreen1 = 0;
            PedestrianGreen2 = 0;
            PedestrianRed = 1;

        }
        counter--; // Decrement counter
    }

    counter = 169;

    while (counter > 160)
    {
        d1 = (counter / 10) % 10;

```

```

    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 1;
        junction1Yellow = 0;
        junction1Red = 0;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;
    }
    counter--; // Decrement counter
}

counter = 159;

while (counter > 150)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 0;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;
    }
}

```

```

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}

counter = 149;

while (counter > 140)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 1;
        junction1Yellow = 0;
        junction1Red = 0;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}

counter = 139;

while (counter > 130)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;

```

```

    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 0;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}

counter = 129;

while (counter > 120)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 1;
        junction1Yellow = 0;
        junction1Red = 0;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
    }
}

```

```

        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}

counter = 119;

while (counter > 110)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 0;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}

counter = 109;

while (counter > 100)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);

```

```

        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 1;
        junction1Yellow = 0;
        junction1Red = 0;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;
    }
    counter--; // Decrement counter
}

// Junction 1: Yellow for 5 seconds

counter = 99;

while (counter > 40)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 1;
        junction1Red = 0;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;
    }
}

```

```

    }
    counter--; // Decrement counter
}

//Junction 1 red for 3 sec

counter = 39;

while (counter > 10)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}
pedestrianButton.enable_irq();
}

void TrafficJunction2On()
// Junction 2: green for 25 sec
{
    pedestrianCrossingActive = true;

    counter = 380;

    while (counter > 130)

```

```

{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 1;
        junction2Yellow = 0;
        junction2Red = 0;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}

}

void TrafficJunction20ff()
{
    pedestrianButton.disable_irq();

    counter = 129;

    while (counter > 120)
    {
        d1 = (counter / 10) % 10;
        d2 = (counter / 100) % 10;
        d3 = (counter / 1000) % 10;
        d4 = (counter / 10000) % 10;
        for (int x = 0; x < 20; x++)
        {
            displaySeg(num[d4], seg[0]);
            displaySeg(num[d3], seg[1]);
            displaySeg(num[d2], seg[2]);
            displaySeg(num[d1], seg[3]);

```

```

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 0;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;
    }
    counter--; // Decrement counter
}

counter = 119;

while (counter > 110)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 1;
        junction2Yellow = 0;
        junction2Red = 0;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;
    }
    counter--; // Decrement counter
}

counter = 109;

```

```

while (counter > 100)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 0;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;
    }
    counter--; // Decrement counter
}

counter = 99;

while (counter > 90)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;
    }
}

```

```

        junction2Green = 1;
        junction2Yellow = 0;
        junction2Red = 0;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}

// Junction 2: Yellow for 5 seconds

counter = 89;

while (counter > 40)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 1;
        junction2Red = 0;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}

//Junction 2 red for 3 sec

```

```

counter = 39;

while (counter > 10)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}

pedestrianButton.enable_irq();
}

void PedestrianOn()
{
    // Pedestrian: Green for 15 seconds

    counter = 209;

    while (counter > 200)
    {
        d1 = (counter / 10) % 10;
        d2 = (counter / 100) % 10;
        d3 = (counter / 1000) % 10;
        d4 = (counter / 10000) % 10;
    }
}

```

```

    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 1;
        PedestrianGreen2 = 0;
        PedestrianRed = 0;

    }
    counter--; // Decrement counter
}

```

```

counter = 199;

```

```

while (counter > 190)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 1;
        PedestrianGreen2 = 1;
    }
}

```

```

        PedestrianRed = 0;

    }
    counter--; // Decrement counter
}

counter = 189;

while (counter > 180)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 1;
        PedestrianRed = 0;

    }
    counter--; // Decrement counter
}

counter = 179;

while (counter > 170)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);

```

```

        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 1;
        PedestrianGreen2 = 0;
        PedestrianRed = 0;
    }
    counter--; // Decrement counter
}

counter = 169;

while (counter > 160)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 1;
        PedestrianGreen2 = 1;
        PedestrianRed = 0;
    }
}

```

```

        counter--; // Decrement counter
    }

    counter = 159;

    while (counter > 150)
    {
        d1 = (counter / 10) % 10;
        d2 = (counter / 100) % 10;
        d3 = (counter / 1000) % 10;
        d4 = (counter / 10000) % 10;
        for (int x = 0; x < 20; x++)
        {
            displaySeg(num[d4], seg[0]);
            displaySeg(num[d3], seg[1]);
            displaySeg(num[d2], seg[2]);
            displaySeg(num[d1], seg[3]);

            junction1Green = 0;
            junction1Yellow = 0;
            junction1Red = 1;

            junction2Green = 0;
            junction2Yellow = 0;
            junction2Red = 1;

            PedestrianGreen1 = 0;
            PedestrianGreen2 = 1;
            PedestrianRed = 0;

        }
        counter--; // Decrement counter
    }

    counter = 149;

    while (counter > 140)
    {
        d1 = (counter / 10) % 10;
        d2 = (counter / 100) % 10;
        d3 = (counter / 1000) % 10;
        d4 = (counter / 10000) % 10;
        for (int x = 0; x < 20; x++)
        {
            displaySeg(num[d4], seg[0]);
            displaySeg(num[d3], seg[1]);
            displaySeg(num[d2], seg[2]);
            displaySeg(num[d1], seg[3]);

```

```

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 1;
        PedestrianGreen2 = 0;
        PedestrianRed = 0;
    }
    counter--; // Decrement counter
}

counter = 139;

while (counter > 130)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 1;
        PedestrianGreen2 = 1;
        PedestrianRed = 0;
    }
    counter--; // Decrement counter
}

```

```

counter = 129;

while (counter > 120)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 1;
        PedestrianRed = 0;
    }
    counter--; // Decrement counter
}

counter = 119;

while (counter > 110)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;

```

```

        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 1;
        PedestrianGreen2 = 0;
        PedestrianRed = 0;

    }
    counter--; // Decrement counter
}

counter = 109;

while (counter > 100)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 1;
        PedestrianGreen2 = 1;
        PedestrianRed = 0;

    }
    counter--; // Decrement counter
}

counter = 99;

while (counter > 90)

```

```

{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 1;
        PedestrianRed = 0;

    }
    counter--; // Decrement counter
}

counter = 89;

while (counter > 80)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;

```

```

        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 1;
        PedestrianGreen2 = 0;
        PedestrianRed = 0;

    }
    counter--; // Decrement counter
}

counter = 79;

while (counter > 70)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;
        junction2Red = 1;

        PedestrianGreen1 = 1;
        PedestrianGreen2 = 1;
        PedestrianRed = 0;

    }
    counter--; // Decrement counter
}

counter = 69;

while (counter > 60)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;

```

```

d3 = (counter / 1000) % 10;
d4 = (counter / 10000) % 10;
for (int x = 0; x < 20; x++)
{
    displaySeg(num[d4], seg[0]);
    displaySeg(num[d3], seg[1]);
    displaySeg(num[d2], seg[2]);
    displaySeg(num[d1], seg[3]);

    junction1Green = 0;
    junction1Yellow = 0;
    junction1Red = 1;

    junction2Green = 0;
    junction2Yellow = 0;
    junction2Red = 1;

    PedestrianGreen1 = 0;
    PedestrianGreen2 = 1;
    PedestrianRed = 0;

}
counter--; // Decrement counter
}

// Pedestrian: Red for 5 seconds

counter = 59;

while (counter > 10)
{
    d1 = (counter / 10) % 10;
    d2 = (counter / 100) % 10;
    d3 = (counter / 1000) % 10;
    d4 = (counter / 10000) % 10;
    for (int x = 0; x < 20; x++)
    {
        displaySeg(num[d4], seg[0]);
        displaySeg(num[d3], seg[1]);
        displaySeg(num[d2], seg[2]);
        displaySeg(num[d1], seg[3]);

        junction1Green = 0;
        junction1Yellow = 0;
        junction1Red = 1;

        junction2Green = 0;
        junction2Yellow = 0;

```

```

        junction2Red = 1;

        PedestrianGreen1 = 0;
        PedestrianGreen2 = 0;
        PedestrianRed = 1;

    }
    counter--; // Decrement counter
}
}

void Pedestrianplay()
{
    if (pedestrianCrossingActive) {
        // Pedestrian button pressed during pedestrian crossing phase at
        TrafficJunction1
        TrafficJunction20ff();
        PedestrianOn();
    } else {
        // Pedestrian button pressed during normal function run
        TrafficJunction10ff();
        PedestrianOn();
    }
}

void junction2()
{
    TrafficJunction10ff();
    TrafficJunction20n();
    TrafficJunction20ff();
}

void normal(float timer)
{
    pedestrianCrossingActive = false;

    junction1Green = 1;
    junction1Yellow = 0;
    junction1Red = 0;

    junction2Green = 0;
    junction2Yellow = 0;
    junction2Red = 1;

    PedestrianGreen1 = 0;
    PedestrianGreen2 = 0;
    PedestrianRed = 1;

    displaySeg(num[0], seg[0]);
}

```

```

    displaySeg(num[0], seg[1]);
    displaySeg(num[0], seg[2]);
    displaySeg(num[0], seg[3]);
}
void obstacleDetection()
{
    float obstacleTimer = 0.0; // Variable to track the duration of obstacle
detection

    while (1)
    {
        // Check if obstacle is detected
        if (sensor.distance() < 7)
        {
            if (obstacleTimer == 0.0)
            {
                carTimer.reset(); // Reset the timer when the obstacle is
first detected
                carTimer.start(); // Start the timer
            }

            // Update the obstacleTimer with the elapsed time
            obstacleTimer = carTimer.read();

            // Check if the obstacle is detected for more than 5 seconds
            if (obstacleTimer >= 5.0)
            {
                junction2(); // Call the junction2 function
                break; // Exit the obstacle detection loop
            }
        }
        else
        {
            obstacleTimer = 0.0; // Reset the obstacleTimer if no obstacle is
detected
        }

        normal(1000.0); // Call the normal function

        wait(1.0); // Delay for 1 second
    }
}

int main()
{
    pedestrianButton.fall(&Pedestrianplay);

```

```
while (1)
{
    {
        normal(1000);
    }

    CarDetection();
    obstacleDetection(); // Call the obstacleDetection function
}
}
```

CONCLUSION

In conclusion, this mini project aimed to design and implement a traffic junction control system with synchronized pedestrian lights. The project successfully achieved its objectives and demonstrated effective control over the traffic flow and pedestrian safety at the junction.

Using hardware components and software programming, the system was able to control the traffic signals at the junction, ensuring smooth and organized traffic movement. The implemented algorithm and timing logic facilitated the sequential switching of the signals, allowing for the efficient allocation of green time to different lanes.

Furthermore, the synchronization of pedestrian lights with the traffic signals enhanced the safety and convenience for pedestrians crossing the junction. The system provided dedicated pedestrian signal phases, ensuring sufficient time for pedestrians to cross safely while coordinating with the traffic flow.

During the testing phase, the system exhibited reliable performance and accurately responded to various scenarios and inputs. The system's ability to transition smoothly between different signal phases and adapt to changing traffic conditions was observed and validated.

While the project was successful overall, some limitations were identified. For instance, the system did not account for real-time traffic detection or adaptive signal timing based on traffic volume. Future enhancements could involve integrating sensors or cameras to gather real-time traffic data and optimize signal timing accordingly.

Overall, this mini project has provided valuable insights into traffic junction control systems and the importance of synchronized signals for ensuring efficient traffic management and pedestrian safety. The knowledge gained from this project can be applied to real-world scenarios, contributing to the development of smarter and more efficient transportation systems.